

Strategic Blueprint for a New Python Agent Framework SDK

I. Introduction & Vision

The rapid evolution of Large Language Models (LLMs) necessitates robust frameworks for building applications that leverage their capabilities. While LLMs excel at text generation and understanding, their true power emerges when they are integrated into systems that can reason, plan, and interact with external tools and data sources – commonly referred to as "agents." This document outlines a comprehensive plan for developing a new Python agent framework Software Development Kit (SDK), informed by an analysis of prominent existing frameworks: Langchain, Agno, the OpenAI Agents SDK, and Google's Agent Development Kit (ADK).

A. Goals for the New Agent SDK

The primary objective of this new SDK is to provide developers with a powerful, flexible, and intuitive toolkit for building sophisticated, production-ready AI agents. Specific goals include:

1. **Accelerated Development:** Enable developers to rapidly prototype, build, and iterate on agentic applications with varying levels of complexity.
2. **Robustness and Scalability:** Provide architectural patterns and components that support the creation of reliable, stateful, and scalable agents suitable for production environments.
3. **Superior Developer Experience (DevEx):** Offer clear abstractions, comprehensive documentation, intuitive APIs, and integrated tools for debugging, evaluation, and monitoring.
4. **Flexibility and Extensibility:** Design a modular architecture that allows easy integration with diverse LLMs, tools, data sources, and memory backends, fostering a rich ecosystem.
5. **Advanced Capabilities:** Natively support complex agent patterns, including multi-agent collaboration, multi-modal interactions (handling text, images, audio, etc.), and sophisticated reasoning workflows.

The envisioned differentiator for this SDK lies in achieving a balance between the comprehensive flexibility seen in frameworks like Langchain and the performance focus or ecosystem integration offered by others like Agno or ADK/OpenAI. It aims to be the framework of choice for developers needing fine-grained control over complex, stateful agent behavior without sacrificing developer productivity or runtime performance.

B. Learning from Existing Frameworks

A critical analysis of existing frameworks provides valuable context for designing the new SDK:

- **Langchain:** Langchain has established itself as a highly comprehensive and modular framework, offering a vast array of abstractions for models, prompts, memory, document handling, and more.¹ Its core strength lies in its composability, particularly through the LangChain Expression Language (LCEL) and the Runnable interface, which provides a standardized way to chain components with built-in streaming and async support.³ For more complex, stateful agents, Langchain introduced LangGraph, a library for building agent workflows as graphs.¹ Langchain's extensive integrations cover hundreds of providers.⁷ However, this breadth can sometimes lead to complexity.¹⁰ Langchain's evolution reflects a move towards greater stability for production use, splitting its original monolithic package into langchain-core (stable base abstractions), langchain (generic cognitive architecture), langchain-community (community integrations), and specific partner packages.² This modularization improves dependency management and maintainability. Furthermore, LangSmith provides essential observability, evaluation, and debugging capabilities, enhancing the developer experience.¹
- **Agno:** Agno positions itself as a lightweight, high-performance alternative, emphasizing simplicity through a "pure Python" approach without complex Domain-Specific Languages (DSLs).¹⁵ It boasts significantly faster agent instantiation times and lower memory footprints compared to alternatives like LangGraph.¹⁵ Agno is designed to be model-agnostic, supporting numerous providers and modalities (text, image, audio, video) natively.¹⁶ Key features include specific modes for multi-agent collaboration (route, collaborate, coordinate)¹⁷, built-in "Agentic RAG" leveraging vector databases¹⁷, dedicated reasoning tools¹⁷, and pluggable memory/storage drivers.¹⁷ Agno also offers supplementary tools like a chat UI²² and an online monitoring platform.¹⁶ It represents a focused approach prioritizing speed and ease of use for specific agentic patterns.
- **OpenAI Agents SDK:** This SDK offers a lightweight framework specifically tailored for building single and multi-agent workflows, primarily within the OpenAI ecosystem.²³ Its core concepts revolve around an Agent definition encompassing instructions, tools, guardrails, and handoffs.²³ It features a built-in agent loop that orchestrates interactions between the LLM and tools.²³ A key feature is the explicit handoff mechanism, enabling agents to delegate tasks to one another.²³ The SDK supports turning Python functions into tools using decorators (@function_tool)²³ and incorporates guardrails for input/output validation.²³

Integrated tracing provides observability into workflow execution.²³ While compatible with models supporting the OpenAI API format²³, its design is naturally aligned with OpenAI's models and features like structured outputs.²³

- **Google Agent Development Kit (ADK):** ADK is presented as an open-source, code-first Python toolkit focused on flexibility, control, and deep integration with the Google Cloud platform, particularly Gemini models and Vertex AI services.²⁹ It allows developers to define agent logic, tools, and orchestration directly in Python. ADK supports distinct agent types, including basic LlmAgent and structured Workflow agents (Sequential, Parallel, Loop).²⁹ It facilitates building hierarchical multi-agent systems.²⁹ The framework provides a rich tool ecosystem, including pre-built tools (e.g., Search, Code Execution), support for custom functions, OpenAPI specifications, and specific Google Cloud tools.²⁹ Context management is handled through a clear separation of Session (current interaction), State (session-specific data), and Memory (searchable, cross-session knowledge).²⁹ ADK emphasizes developer experience with features like a web-based UI (adk web) for interactive testing and debugging, and a built-in evaluation framework (adk eval).³⁴

The landscape of agent frameworks reveals a spectrum of design choices. Langchain prioritizes modularity and a vast, integration-rich ecosystem, evolving towards better production stability through package separation.² Agno carves a niche with its focus on performance, simplicity, and native multi-modal support.¹⁵ OpenAI's SDK provides a streamlined experience for multi-agent workflows tightly coupled with its API.²³ Google's ADK offers fine-grained control and deep integration within the Google Cloud ecosystem.²⁹ Understanding these differing philosophies and their resultant trade-offs (e.g., flexibility vs. performance, broad compatibility vs. ecosystem depth) is fundamental to defining the direction of the new SDK.

C. Proposed Architectural Philosophy

Based on the stated goals and the analysis of existing frameworks, the proposed architectural philosophy for the new SDK is **Modular Control with Enhanced Developer Experience**. This philosophy entails:

1. **Modular Core:** Adopt a layered package structure similar to Langchain's recent refactoring², separating stable core abstractions from evolving integrations and optional developer tools. This promotes maintainability, stability, and a lean core.
2. **Explicit Control:** Embrace explicit, graph-based orchestration inspired by LangGraph² and ADK Workflows²⁹ as the primary mechanism for defining complex agent logic and state transitions. This provides developers with the necessary control for building robust, non-trivial agents.

3. **Pythonic Interface:** Prioritize clear, intuitive APIs that adhere to standard Python conventions, drawing inspiration from Agno's simplicity ¹⁵ and ADK's code-first approach ³⁰, while potentially offering declarative syntax sugar (like LCEL ⁶) for common patterns.
4. **Developer Experience Focus:** Integrate observability (tracing, logging) and evaluation frameworks as first-class citizens, learning from LangSmith ¹⁴ and ADK's DevEx tools.³⁶ Comprehensive documentation and examples are paramount.
5. **Extensibility and Agnosticism:** Design core interfaces (Models, Tools, Memory) to be easily extensible, fostering a vibrant ecosystem, while striving for model and vendor agnosticism where feasible, similar to Langchain ¹² and Agno.¹⁵

This philosophy aims to combine the flexibility and ecosystem potential of Langchain with the control offered by graph-based systems like LangGraph and ADK, the developer-friendliness of Agno and ADK, and the production-readiness focus evident in LangSmith and ADK's evaluation tools.

II. Core SDK Architecture

Building upon the chosen philosophy, the core architecture will emphasize modularity, clear design principles, and a structure that facilitates both ease of use and advanced customization.

A. Package Structure

Learning from Langchain's evolution towards a more manageable structure for production environments ², the new SDK will adopt a multi-package architecture:

1. **sdk-core:**
 - **Purpose:** The foundational layer containing stable, base abstractions and interfaces with minimal external dependencies.
 - **Contents:** Interfaces for BaseRunnable (defining invoke, stream, batch), BaseLanguageModel, BaseChatModel, BaseMultiModalModel, BaseTool, BasePromptTemplate, BaseMemory, BaseChatMessageHistory, BaseDocumentLoader, BaseTextSplitter, BaseEmbeddings, BaseVectorStore. It will also contain core data structures (e.g., Document, Message types) and potentially basic, in-memory implementations of some interfaces (e.g., InMemoryChatMessageHistory).
 - **Analogy:** Similar role to langchain-core.
2. **sdk-agents:**
 - **Purpose:** Contains the primary logic for agent orchestration and state

management. Depends only on sdk-core.

- **Contents:** The graph-based orchestration engine (e.g., StateGraph), implementations for managing state transitions, standard control flow nodes (sequence, parallel, conditional, loop), core memory implementations (e.g., BufferMemory, TokenBufferMemory), and potentially definitions for standard agent types (e.g., a basic ReAct agent).
 - **Analogy:** Combines concepts from langchain (cognitive architecture) and langgraph (orchestration).
3. **sdk-integrations:**
- **Purpose:** Houses third-party integrations for models, vector stores, tools, memory backends, etc. All dependencies within this package will be optional.
 - **Contents:** Modules for specific vendors (e.g., sdk-integrations-openai, sdk-integrations-anthropic, sdk-integrations-googleai, sdk-integrations-weaviate). Each module implements the relevant interfaces defined in sdk-core.
 - **Analogy:** Similar to langchain-community and Langchain's partner packages (e.g., langchain-openai).⁷
4. **sdk-devtools:**
- **Purpose:** Provides optional tools to enhance the developer experience, potentially with additional dependencies.
 - **Contents:** Utilities for tracing export (e.g., OpenTelemetry exporters), evaluation helpers (metric calculators, dataset loaders), potentially a simple CLI or UI components for debugging or interaction (inspired by ADK's adk web ³⁶).
 - **Analogy:** Analogous to aspects of LangSmith ¹⁴ or ADK's CLI/UI tools.³⁴

This structure ensures a lightweight core, isolates dependencies, allows independent versioning of integrations, and provides dedicated space for developer tooling.

B. Key Design Principles

The SDK's design will adhere to the following principles:

1. **Pythonic Interface:** APIs should feel natural to Python developers. While powerful composition syntax (like LCEL's `|` operator ⁵) can be offered for convenience, the underlying mechanisms should be accessible via standard Python methods and classes. This avoids the potential learning curve and "un-Pythonic" feel some users associate with DSLs ⁵, aligning more with Agno's philosophy ¹⁵ and ADK's code-first approach.²⁹
2. **Composability (via Runnables):** Adopt a standard interface, analogous to Langchain's Runnable ³, for all core components (models, prompts, tools,

retrievers, agent nodes). This interface should define standard methods like `invoke`, `stream`, `batch`, and their asynchronous counterparts (`ainvoke`, `astream`, `abatch`).³ This ensures components can be easily chained and integrated within the orchestration engine.

3. **Extensibility:** Core interfaces (`BaseModel`, `BaseTool`, `BaseMemory`, etc.) must be designed for easy implementation by external developers or partners. Clear documentation and simple base classes will encourage community contributions and ecosystem growth, a key factor in Langchain's success.¹²
4. **Explicitness over Implicitness:** For agent orchestration and state management, the design will favor explicit definition. Developers should clearly define the nodes, edges, and state transitions in their agent graphs, similar to LangGraph³⁷ and ADK's workflow agents.²⁹ While this might involve slightly more setup for simple cases compared to implicit loops (like OpenAI's Agent Loop²³) or legacy chains³⁹, it provides necessary clarity and control for complex, reliable agents. Simple abstractions or pre-built graphs can be provided for common use cases.
5. **Performance Awareness:** While flexibility is key, design decisions should consider performance implications. Efficient state management, native async support, and optimized parallel execution (where applicable, like LCEL `RunnableParallel`⁶) are important considerations, drawing inspiration from Agno's focus.¹⁵

C. Framework Architectural Comparison

To solidify the strategic positioning, the following table compares the high-level architectural approaches of the researched frameworks:

Framework	Core Philosophy	Orchestration Style	State Management	Multi-Agent Approach	Key Strengths	Key Weaknesses/Trade-offs
Langchain	Modularity, Composability	LCEL (Declarative DAGs), LangGraph (Graphs)	LangGraph, StateGraph	LangGraph	Breadth, Flexibility, Ecosystem, DevEx (LangSmith)	Potential Complexity, Evolving Architecture
Agno	Simplicity, Performance	Pythonic Loops, Agent	Storage/Memory Drivers	Route, Collaborate,	Speed, Low Footprint,	Newer, Smaller Ecosystem

		Teams		Coordinate	Multi-modal, Simplicity	, Less Control?
OpenAI SDK	Ecosystem Integration (OpenAI)	Built-in Agent Loop	Implicit within Loop?	Handoffs	Simplicity, OpenAI Focus, Handoffs	Limited outside OpenAI, Less Explicit Control
Google ADK	Control, GCP Integration	Code-first, Workflow Agents (Graph-like)	Session, State, Memory	Hierarchies, Delegation	Control, DevEx (UI/Eval), GCP Focus, Structure	Newer, GCP-centric

Data synthesized from: ²

This comparison highlights the distinct approaches. Langchain offers the most general-purpose, flexible system, especially with LangGraph. Agno targets performance and simplicity. OpenAI provides a streamlined path for multi-agent systems using its models. ADK offers deep control and tight integration with Google Cloud. The new SDK aims to carve a niche by combining Langchain/LangGraph's control and flexibility with Agno/ADK's Pythonic feel and a strong DevEx focus inspired by LangSmith and ADK.

III. Foundational SDK Abstractions

The sdk-core package will define the essential interfaces and data structures upon which the rest of the framework is built. These abstractions must be stable, minimal, and clearly defined to ensure interoperability and extensibility.

A. Model Interface

Standardizing interaction with various language models is crucial.

- **Base Classes:**
 - BaseLanguageModel: Interface for models primarily dealing with text-in, text-out operations. While potentially less common for new applications, it provides compatibility.³
 - BaseChatModel: The standard interface for modern conversational models,

accepting a list of Message objects and returning an AIMessage.³

- **BaseMultiModalModel:** An interface extending BaseChatModel to explicitly handle multi-modal inputs (e.g., images, audio passed alongside text) and potentially generate multi-modal outputs, drawing inspiration from Agno's native support.¹⁷
- **Standard Parameters:** Model initializers should accept standardized parameters where applicable across providers, such as model_name, temperature, max_tokens, api_key, base_url, request_timeout, max_retries, etc..³ Provider-specific parameters can be passed via model_kwargs.
- **Input/Output Types:** Define clear Pydantic models for messages (e.g., BaseMessage, HumanMessage, AIMessage, SystemMessage, ToolMessage) similar to langchain-core.³ AIMessage should accommodate tool calls and potentially multi-modal content.

B. Tool Definition and Integration Strategy

Tools empower agents to interact with the world.

- **BaseTool Class:** Define a BaseTool class inheriting from BaseRunnable. It must include:
 - name (str): Unique identifier for the tool.
 - description (str): Natural language description used by the LLM to select the tool. Clarity is essential.⁴¹
 - args_schema (Pydantic BaseModel): Defines the expected input arguments and their types, enabling validation and structured invocation.⁴¹
 - _run / _arun methods: Synchronous and asynchronous methods containing the tool's execution logic.⁴¹
- **Function Tools:** Provide a decorator (e.g., @tool) to easily convert Python functions into BaseTool instances, automatically inferring name, description (from docstring), and args_schema (from type hints). This pattern improves developer experience, as seen in OpenAI SDK²³ and ADK.²⁹
- **Toolkits:** Consider supporting the concept of Toolkits – collections of related tools that can be passed together to an agent.⁴³ This can simplify configuration.
- **OpenAPI Integration:** Plan for a mechanism to create tools directly from OpenAPI specifications, allowing agents to interact with existing REST APIs.²⁹ This could be a utility function or a specific Tool subclass.
- **Invocation:** The orchestration engine (Section IV) will be responsible for parsing tool call requests from the LLM, validating inputs against the args_schema, invoking the tool's _run/_arun method, and formatting the output back into a ToolMessage.

C. Prompt Management System

Effective prompting is key to agent performance.

- **Template Classes:**
 - `StringPromptTemplate`: For simple string formatting.³
 - `ChatPromptTemplate`: For constructing lists of `Message` objects, supporting roles (system, user, ai, tool) and variable substitution.³
- **Dynamic Content:** Include a mechanism similar to Langchain's `MessagesPlaceholder`¹² to dynamically insert lists of messages (e.g., chat history, tool results) into a `ChatPromptTemplate`.
- **Advanced Features (Future Consideration):** Explore features inspired by LangSmith's Hub¹² or Prompt Engineering tools¹⁴, such as prompt versioning, sharing, A/B testing, or integration with prompt optimization techniques, potentially within `sdk-devtools`.

D. Data Handling Primitives

Standardized handling of documents and vector data is essential for RAG and knowledge-grounded agents.

- **Document:** A core class representing a piece of text, containing `page_content` (str) and `metadata` (dict).³ Consider adding an optional `id` field.⁴⁶
- **BaseDocumentLoader:** Interface for loading `Document` objects from various sources (files, databases, APIs).³ Specific loader implementations belong in `sdk-integrations`.
- **BaseTextSplitter:** Interface for splitting `Document` objects into smaller chunks based on different strategies (character, token, recursive).³ Core splitting logic can reside in `sdk-core`, while token-based splitters might depend on specific tokenizers in `sdk-integrations`.
- **BaseEmbeddings:** Interface for models that generate vector embeddings for text (or other modalities).³ Implementations belong in `sdk-integrations`.
- **BaseVectorStore:** Interface defining methods for adding `Document` objects (with embeddings) and performing similarity searches (e.g., `similarity_search`, `max_marginal_relevance_search`).³ Implementations for specific vector databases (Chroma, Milvus, Pinecone, etc.) belong in `sdk-integrations`. The design should consider potential improvements like those planned for Langchain Vectorstores V2.¹¹
- **BaseRetriever:** A more general interface than `BaseVectorStore`, representing any component that retrieves relevant `Document` objects given a query.³ Vector stores can be wrapped as retrievers, but retrievers could also query traditional

databases or search engines.

Establishing these well-defined, stable abstractions in sdk-core is paramount. Langchain's experience demonstrates that a solid core fosters a thriving ecosystem by providing clear interfaces for community and partner integrations.² This foundation enables developers to confidently build upon the SDK, knowing the fundamental building blocks are reliable and interoperable.

IV. Agent Orchestration Engine

The orchestration engine, residing primarily in the sdk-agents package, dictates how agent components (LLMs, tools, memory) are coordinated to execute tasks. Its design significantly impacts the SDK's capabilities and complexity.

A. Core Execution Model

The choice of execution model determines the fundamental control flow paradigm.

- **Simple Loop:** A basic cycle (LLM decides action -> execute action -> observe result -> repeat) is easy to understand but struggles with complex logic, branching, and state management. This is the model used by the OpenAI Agents SDK.²³
- **Declarative Composition (LCEL-like):** Langchain's LCEL uses primitives like RunnableSequence and RunnableParallel with a pipe syntax (|) to compose linear or DAG-based workflows.⁵ It excels at streaming and async operations for these structures but isn't inherently designed for cyclic graphs or complex state transitions needed in many agentic systems.⁶
- **Explicit Graph-Based:** Modeling the workflow as a graph, where nodes represent computational steps (LLM calls, tool executions) and edges represent transitions, offers maximum flexibility. This approach, adopted by LangGraph² and ADK's Workflow agents²⁹, naturally handles cycles, conditional branching, state management, and complex multi-step reasoning.

Recommendation: Implement an **explicit graph-based model** as the core orchestration engine. This provides the necessary power and flexibility for building sophisticated, stateful agents. The complexity observed in real-world agent applications often necessitates cycles (e.g., retrying a tool call, iterative refinement) and conditional logic that are difficult to express cleanly in purely linear or DAG structures. While a graph model might have a slightly steeper initial learning curve, its ability to handle diverse control flows makes it a more robust foundation. Simpler LCEL-like syntax sugar can be provided on top of the graph engine for common

sequential or parallel patterns to improve ease of use for simpler cases.

B. State Management Design

A graph-based engine requires explicit state management.

- **State Representation:** Define a central state object (e.g., a Pydantic model or TypedDict) that is passed between nodes in the graph. This state object holds all relevant information for the execution flow, such as input queries, intermediate results, chat history, tool outputs, and any other contextual data. LangGraph uses StateGraph parameterized by a state definition³⁷, while ADK uses Session and State objects.³¹
- **State Updates:** Nodes in the graph should return updates to the state object. Define clear mechanisms for how these updates are applied – for example, allowing nodes to either overwrite specific state attributes or append to list-based attributes (like LangGraph's operator.add annotation).³⁷
- **Persistence:** The state object should be serializable to allow for persistence, enabling workflows to be paused, resumed, and potentially span long durations (See Section V).

C. Control Flow Mechanisms

The graph engine must support various control flow patterns:

- **Sequential Execution:** Represented by direct edges connecting nodes in the graph.
- **Parallel Execution:** A dedicated node type (similar to LCEL's RunnableParallel⁶) or graph structure (like ADK's Parallel agent²⁹) should allow multiple nodes or branches to execute concurrently with the same input state, merging their results back into the state.
- **Conditional Branching:** Edges should be configurable to direct the flow based on the current state. A node's output could determine which edge is taken next (e.g., based on whether an LLM decided to call a tool or provide a final answer, similar to LangGraph's conditional edges⁵¹).
- **Loops/Cycles:** The graph structure inherently supports cycles, allowing for iterative processes, retries, or multi-turn interactions with tools or sub-agents.⁵¹ ADK also provides an explicit Loop agent type.²⁹

D. Asynchronous and Streaming Support Strategy

Modern AI applications require responsive interfaces and efficient resource utilization.

- **Async-First Design:** All core components of the orchestration engine, including

node execution and state transitions, must support asynchronous operation (async/await). This aligns with the principles successfully implemented in LCEL³ and is crucial for I/O-bound tasks like API calls to LLMs or tools.

- **First-Class Streaming:** Streaming should be a primary consideration. The engine should support streaming final outputs token-by-token from LLM nodes.³ Furthermore, consider supporting streaming of intermediate steps or events (like LangGraph's `astream_log` or `astream_events`³, or ADK's event system³⁴), which provides visibility into the agent's reasoning process and improves perceived performance in user interfaces.⁵⁵

The evolution of frameworks like Langchain from simpler chain abstractions towards graph-based systems like LangGraph² underscores the need for explicit control flow and state management in building robust agents. ADK's inclusion of distinct workflow agents²⁹ further reinforces this trend. By adopting a graph-based core, the new SDK can effectively handle the complexity inherent in agentic workflows involving loops, conditional logic, and state persistence, setting a strong foundation for advanced applications.

V. Memory Architecture

Effective memory management is crucial for agents to maintain context, learn from past interactions, and provide coherent, personalized experiences. The SDK's memory architecture must be flexible and support various strategies.

A. Memory Interface Design

Standardized interfaces allow for different memory implementations and storage backends.

- **BaseMemory Interface:** Defines the core contract for memory components integrated into the agent orchestration loop.⁵⁷ Key methods should include:
 - `load_memory_variables(inputs: dict) -> dict`: Retrieves relevant context from memory to be injected into the agent's prompt or state based on current inputs.
 - `save_context(inputs: dict, outputs: dict) -> None`: Updates the memory with the latest interaction (inputs and agent's outputs).
 - Asynchronous versions (`aload_memory_variables`, `asave_context`) must also be defined.
- **BaseChatMessageHistory Interface:** Specifically handles the storage and retrieval of sequences of Message objects.⁵⁷ This allows decoupling the chat history storage mechanism (e.g., in-memory list, database, file) from the memory

components that might consume this history (like buffer or summary memory). Methods should include `add_message`, `get_messages`, `clear`.

B. Supported Memory Types & Strategies

Different use cases demand different memory strategies. The SDK should support a range:

- **Core Implementations (`sdk-core` / `sdk-agents`):**
 - `BufferMemory`: Stores the full conversation history as a list of messages. Simple but can exceed context limits.⁵⁸
 - `BufferWindowMemory`: Stores only the last K conversation turns.⁵⁸
 - `TokenBufferMemory`: Stores recent messages, automatically pruning older ones to stay below a specified token limit.⁵⁸
- **Advanced Implementations (Potentially `sdk-agents` or `sdk-integrations`):**
 - `SummaryMemory`: Uses an LLM to progressively summarize the conversation, keeping the context concise.⁵⁸ Requires an LLM dependency.
 - `SummaryBufferMemory`: Combines a buffer of recent messages with a running summary of older messages.⁵⁸ Balances recency and context length.
 - `VectorStoreMemory`: Embeds conversation turns and stores them in a vector store. Retrieves relevant past interactions based on semantic similarity to the current input.⁵⁸ Requires embedding model and vector store dependencies.
 - `EntityMemory`: Extracts key entities (people, places, concepts) mentioned in the conversation and stores information related to them.⁵⁸ Requires an LLM for entity extraction.
- **Comparison with ADK:** Google ADK distinguishes between Session (current interaction history/events), State (temporary data within the session), and Memory (searchable, persistent, cross-session knowledge base).³¹ This separation is conceptually clear. The proposed SDK can achieve similar separation by using `ChatMessageHistory` for session events, the graph's State object for session-specific temporary data, and dedicated Memory components (like `VectorStoreMemory`) for persistent, searchable knowledge.

C. Persistence and Storage Drivers

For real-world applications, memory needs to persist beyond a single execution.

- **Storage Abstraction:** Define a `BaseStore` or similar interface for key-value storage, and potentially specialized interfaces for chat history (`BaseChatMessageHistory` can serve this role) or vector data (`BaseVectorStore`).
- **Driver Implementations:**
 - Provide basic, non-persistent drivers in `sdk-core` (e.g., `InMemoryStore`,

InMemoryChatMessageHistory).

- Implement persistent drivers in sdk-integrations for common backends:
 - Chat History: File-based, Redis, SQL databases (Postgres, SQLite), etc. Agno examples show SqliteAgentStorage which seems to handle session persistence including history.⁶⁰
 - Vector Memory: Integrations with vector databases (Weaviate, Pinecone, Chroma, LanceDB, etc.).²⁰
 - General State/KV: Redis, SQL databases. ADK uses SessionService and MemoryService abstractions with implementations like InMemorySessionService or the persistent VertexAiRagMemoryService.³¹
- **Configuration:** Memory components should be configurable to use specific storage drivers for persistence.

D. Memory Strategy Comparison

The choice of memory strategy involves trade-offs.

Framework	Memory Type/Concept	Description/Use Case	Persistence	Key Parameters/Dependencies
Langchain	BufferMemory	Stores full history; simple, good for short chats.	Via ChatHistory	chat_memory (ChatHistory backend)
	BufferWindowMemory	Stores last K turns; prevents context overflow.	Via ChatHistory	k, chat_memory
	TokenBufferMemory	Stores history up to token limit; manages context window size.	Via ChatHistory	llm, max_token_limit, chat_memory
	SummaryMemory	Summarizes history; good for long chats, maintains core context.	Via ChatHistory	llm, chat_memory

	VectorStoreMemory	Retrieves relevant past turns via similarity search; RAG-like recall.	Vector Store	retriever (VectorStoreRetriever)
	EntityMemory	Tracks key entities; structured memory of discussed subjects.	Via ChatHistory	llm, entity_store
Agno	Memory / Storage Drivers	Pluggable drivers for session state & long-term memory.	Via Storage Drivers	storage (e.g., SqliteAgentStorage)
Google ADK	Session State	Temporary key-value data within the current conversation thread.	Via SessionService	-
	MemoryService (InMemory)	In-memory store for cross-session knowledge; basic keyword search.	No	-
	MemoryService (Vertex AI RAG)	Persistent, scalable knowledge store using Vertex AI RAG; semantic search.	Yes (Vertex AI)	GCP Project, RAG Corpus Resource

Data synthesized from: ¹⁷

This comparison underscores the need for a flexible memory system. Simple

applications might only need buffer memory, while complex agents requiring long-term recall or dealing with extensive conversations will benefit from summarization or vector-based retrieval. Providing interfaces for persistence (like BaseChatMessageHistory and potentially BaseStore) allows developers to plug in appropriate backends (in-memory for testing, databases for production) without changing the core agent logic. ADK's explicit separation of session state and long-term memory³¹ offers a useful conceptual model to consider.

VI. Multi-Agent Capabilities

Building systems where multiple agents collaborate requires dedicated architectural support for communication, coordination, and task delegation.

A. Designing for Agent Collaboration

The SDK needs to define how agents interact within a larger system.

- **Communication Mechanisms:**
 - **Shared State:** Agents can interact implicitly by reading and modifying a shared state object managed by the orchestration engine (common in LangGraph³⁷). Requires careful state design to avoid conflicts.
 - **Direct Calls/Handoffs:** One agent explicitly invokes or transfers control to another agent. This is the primary mechanism in the OpenAI SDK (handoffs)²³ and is supported in ADK hierarchies.²⁹ Requires defining how context/state is passed during the handoff.
 - **Message Passing/Event Bus:** Agents could communicate by sending messages to each other via a dedicated bus. This offers loose coupling but adds complexity. (Less common in the researched frameworks).
- **Interaction Patterns:** Support for various patterns is desirable:
 - **Delegation/Handoff:** Implement a mechanism similar to OpenAI's handoff tool²⁷, allowing an agent node in the graph to transition execution to a different agent graph, potentially passing specific inputs.
 - **Coordination (Supervisor/Worker):** Allow a "supervisor" agent graph to invoke "worker" sub-agent graphs and integrate their results. The supervisor node would manage the state and decide which sub-agent to call next. This aligns with Agno's coordinate mode¹⁷ and ADK's hierarchical approach.²⁹
 - **Collaboration (Shared Workspace):** Enable multiple agents to operate on a shared state object within the graph, potentially taking turns or working in parallel on different aspects of the state. This reflects Agno's collaborate mode¹⁷ and can be implemented using shared state in the graph.
 - **Routing:** Use conditional edges in the main graph, controlled by an agent's

output, to route the workflow to different specialized agents or sub-graphs, similar to Agno's route mode ¹⁷ and achievable with LangGraph's conditional edges.⁵¹

Recommendation: Leverage the flexibility of the graph-based orchestration engine. Handoffs can be implemented as special edge types or node outputs that trigger transitions to different graphs. Coordination can be achieved by having supervisor nodes call sub-graphs. Collaboration can occur via shared state modification within a single graph. Routing is handled by conditional edges.

B. Team Structures and Management

Provide ways to organize and manage groups of collaborating agents.

- **Agent Grouping:** Allow developers to logically group agents (e.g., as an AgentTeam class or similar concept, inspired by Agno ¹⁷ and ADK's multi-agent systems ²⁹).
- **Configuration:** Define how teams are configured – do they share memory, tools, or state? How is the interaction mode (e.g., coordination, routing) specified for the team?
- **Execution:** Define how an AgentTeam is executed – likely by invoking a primary coordinating or routing agent within the team structure.

C. Tool Integration Approaches

The way tools are defined and integrated impacts how easily agents can leverage external capabilities.

Framework	Tool Definition	Input Schema	Execution Integration	Key Features
Langchain	BaseTool class	Pydantic	AgentExecutor / LangGraph Node	Async, Error Handling, Toolkits, Structured Tool
Agno	Python function or Toolkit class	Type Hints	Agent Loop / Tool Call	Simple, Pythonic, ReasoningTools
OpenAI SDK	@function_tool decorator	Type Hints/Pydantic	Agent Loop / Tool Call	Simple Decorator, Auto-Schema,

				Validation
Google ADK	Python function (@tool)	Type Hints	Runner / Agent Loop / Tool Call	Simple Decorator, OpenAPI, GCP Tools

Data synthesized from: ¹⁷

The trend favors simplifying tool creation using decorators on standard Python functions, as seen in OpenAI SDK and ADK. This lowers the barrier to entry compared to subclassing BaseTool. However, Langchain's BaseTool offers more structure and potentially more explicit control over aspects like error handling or callbacks within the tool itself. Agno also supports simple functions.

Recommendation: Support both approaches. Provide a @tool decorator for easily converting functions (inferring schema from type hints and docstrings) and a BaseTool class for more complex scenarios requiring finer control over execution or configuration. Ensure seamless integration of both types into the graph orchestration engine.

VII. Developer Experience and Ecosystem

A successful SDK requires more than just functional components; it needs excellent developer experience (DevEx) features and fosters a supportive ecosystem.

A. SDK Usability and Documentation Plan

Clear documentation and intuitive APIs are non-negotiable.

- **Documentation Structure:** Organize documentation logically with:
 - **Getting Started:** Quick installation and simple "hello world" agent examples.
 - **Tutorials:** Step-by-step guides for building common agent types (e.g., RAG agent, tool-using agent, multi-agent system). Langchain ¹ and ADK ²⁹ provide good models.
 - **How-To Guides:** Focused recipes for specific tasks (e.g., adding memory, creating custom tools, deploying). Langchain's structure is effective.¹
 - **Conceptual Guides:** Explanations of core concepts (agents, state, memory, orchestration, tools). Langchain ¹ and ADK ²⁹ offer these.
 - **API Reference:** Detailed documentation for all public classes and functions, ideally auto-generated from code. Langchain ⁷ and ADK ²⁹ provide

comprehensive references.

- **Examples:** Provide a rich library of runnable examples covering diverse use cases and integrations, similar to Langchain Academy ⁶⁵, Agno's cookbook ¹⁵, and ADK Samples.⁶⁷
- **API Design:** Ensure APIs are consistent, predictable, and leverage Python conventions. Provide clear error messages to aid debugging.

B. Observability: Tracing and Debugging Integration

Understanding agent behavior is critical due to their non-deterministic nature.¹⁴

- **Built-in Tracing:** Implement tracing throughout the SDK's core components (orchestration engine, model calls, tool calls, memory access). Capture inputs, outputs, errors, latency, and intermediate steps.
- **Standard Formats:** Natively support exporting traces using the OpenTelemetry standard.⁶⁹ This allows integration with various observability platforms (LangSmith, Datadog, Jaeger, etc.). LangSmith's OTel support provides a reference.⁶⁸
- **Visualization & Debugging:**
 - Provide utilities for visualizing agent traces (e.g., printing a structured log).
 - Consider a simple web UI for visualizing graph execution and state changes, inspired by ADK's adk web ³⁵ or LangGraph Studio.⁵⁵ This could be part of sdk-devtools.
 - Ensure easy integration with dedicated platforms like LangSmith ¹⁴ or others mentioned in OpenAI's tracing documentation.²³ ADK also logs detailed events.³⁴

C. Evaluation Framework Design

Systematic evaluation is necessary to ensure agent quality and reliability.

- **Core Components (sdk-devtools):**
 - **Dataset Definition:** Define formats for evaluation datasets (e.g., JSON files containing inputs, expected intermediate steps like tool calls, expected final outputs, reference contexts). ADK's test files and evalsets offer a good starting point.³⁶
 - **Metrics:** Implement or provide interfaces for common evaluation metrics:
 - Response quality (e.g., ROUGE, BLEU, BERTScore, relevance).
 - Tool usage accuracy (comparing actual vs. expected tool calls, like ADK's tool_trajectory_avg_score ³⁶).
 - LLM-as-Judge evaluators (using an LLM to score correctness, relevance, safety, etc., as offered by LangSmith ¹³).

- **Evaluation Runners:** Provide ways to execute evaluations:
 - Programmatic API (e.g., a function like `evaluate(agent, dataset)`). ADK allows programmatic runs via `AgentEvaluator.evaluate`.³⁶
 - CLI tool (e.g., `sdk eval <agent> <dataset>`), similar to `adk eval`.³⁶
 - Integration with the Dev UI (like ADK's `adk web Eval` tab ³⁶).

D. Deployment Strategy Considerations

While the core SDK should remain deployment-agnostic, providing guidance and tools simplifies productionization.

- **Containerization:** Offer example Dockerfiles and best practices for packaging agents as container images.
- **API Serving:** Consider a dedicated package (e.g., `sdk-serve`) analogous to `LangServe`² for quickly deploying agents as REST APIs. Acknowledge that simple REST wrappers might be insufficient for complex, stateful graph-based agents requiring long-running processes or persistent connections (a limitation noted for `LangServe` vs. `LangGraph Platform`²).
- **Guidance:** Provide documentation on deploying to common platforms like serverless functions (e.g., AWS Lambda, Google Cloud Functions), container orchestration platforms (Kubernetes, GKE⁷⁰), managed compute (Cloud Run⁷⁰), or dedicated agent platforms (Vertex AI Agent Engine²⁹).

E. Developer Experience Feature Comparison

Strong DevEx is a competitive advantage.

Feature	Langchain	Agno	OpenAI SDK	Google ADK	Proposed SDK Target
Tracing/Debugging	LangSmith (External)	agno.com (External), Logs	Built-in Tracing	Built-in Events/Callbacks, <code>adk web UI</code>	Built-in OTel Tracing, Optional Dev UI
Evaluation	LangSmith (External)	Monitoring via agno.com	Via Tracing/External	<code>adk eval CLI</code> , <code>adk web UI</code> , Programmatic	CLI, Programmatic, Optional Dev UI
Deployment Support	LangServe, LangGraph	Via FastAPI/Cont	Via Containers	Agent Engine, Cloud Run,	Guidance, Optional <code>sdk-serve</code>

	Platform	ainers		Containers	package
Documentati on	Extensive, Multi-format	Good, Growing	Good, Focused	Extensive, Multi-format	Extensive, Multi-format, Rich Examples
Community	Very Large, Active	Growing	Growing	Growing, GCP-focuse d	Foster Actively

Data synthesized from: ¹

The analysis reveals that while all frameworks offer some DevEx features, Langchain (via LangSmith) and Google ADK set a high standard for integrated tracing, evaluation, and debugging tools.¹⁴ Agno provides monitoring¹⁶ and OpenAI has built-in tracing.²³ For the new SDK to be competitive, particularly for building production systems, incorporating robust observability and evaluation capabilities from the outset is crucial. This reduces developer friction in understanding and improving complex agent behavior.

VIII. Synthesis and Recommendations

This plan outlines a strategy for building a new Python agent framework SDK that learns from existing solutions while carving out its own identity.

A. Key Architectural Decisions Summarized

- **Philosophy:** Modular Control with Enhanced Developer Experience.
- **Structure:** Multi-package (sdk-core, sdk-agents, sdk-integrations, sdk-devtools).
- **Core Abstractions:** Standardized interfaces for Models (Chat, Multi-modal), Tools (BaseTool + @tool decorator), Prompts, Memory, and Data Primitives (Document, VectorStore, etc.).
- **Orchestration:** Explicit, graph-based engine (sdk-agents) supporting complex control flow (sequence, parallel, conditional, loops) with built-in state management.
- **Memory:** Flexible architecture with core types (Buffer, TokenBuffer) and interfaces for advanced strategies (Summary, Vector) and persistence via storage drivers.
- **Multi-Agent:** Support for delegation (handoffs), coordination, collaboration, and routing via graph structures and team management concepts.

- **Developer Experience:** Prioritize Pythonic APIs, comprehensive documentation/examples, integrated OpenTelemetry tracing, and a built-in evaluation framework.

B. Recommended Feature Set for Initial Version (MVP)

To deliver value quickly and allow for iterative development, the Minimum Viable Product (MVP) should focus on establishing the core architecture and essential functionality:

1. **sdk-core:** Define and implement all base interfaces (BaseRunnable, BaseChatModel, BaseTool, BaseMemory, BaseChatMessageHistory, Document, etc.) and basic data structures. Include InMemoryChatMessageHistory.
2. **sdk-agents:** Implement the core graph-based orchestration engine (StateGraph) with support for sequential execution, conditional branching, and basic state management (overwrite/append). Include BufferMemory and TokenBufferMemory.
3. **sdk-integrations:** Provide integrations for at least one major LLM provider (e.g., OpenAI via langchain-openai patterns), one vector store (e.g., Chroma, LanceDB), and a simple tool (e.g., DuckDuckGo search via Agno/Langchain patterns).
4. **Tooling:** Implement the @tool decorator for easy function-to-tool conversion.
5. **Observability:** Basic OpenTelemetry trace generation for major events (agent start/end, node execution, LLM calls, tool calls).
6. **Documentation:** Foundational "Getting Started" guide, tutorials for a simple agent and a RAG agent, and API reference for core components.

This MVP provides a functional core upon which more advanced features and integrations can be built.

C. Potential Trade-offs and Future Considerations

- **Trade-offs:**
 - The initial adoption of a graph-based engine might present a slightly steeper learning curve than simpler loop-based frameworks for basic agents. This can be mitigated by providing clear tutorials and potentially helper functions or pre-built graphs for common patterns.
 - Prioritizing control and flexibility might initially lead to slightly lower out-of-the-box performance compared to a highly optimized, less flexible framework like Agno. Performance tuning can be addressed in later iterations.
- **Future Considerations:**
 - **Advanced Memory:** Implement SummaryMemory, VectorStoreMemory, EntityMemory.
 - **Multi-Agent:** Add explicit AgentTeam structures and refined patterns for

coordination/collaboration.

- **Multi-Modal:** Fully implement BaseMultiModalModel and support image/audio/video handling in messages and tools.
- **Evaluation:** Enhance the evaluation framework with more metrics, LLM-as-judge capabilities, and potentially a dedicated UI in sdk-devtools.
- **Deployment:** Develop the sdk-serve package for easy REST deployment.
- **Ecosystem:** Actively foster community contributions for integrations.
- **LCEL-like Syntax:** Add optional declarative syntax sugar for composing simple graph structures.
- **Retries/Fallbacks:** Implement robust retry (with_retry) and fallback (with_fallbacks) mechanisms for Runnable, drawing from Langchain's implementation.⁷¹

IX. Conclusion

A. Summary of the Proposed Plan

This document presents a strategic blueprint for developing a new Python agent framework SDK. The plan advocates for a **Modular Control with Enhanced Developer Experience** philosophy, learning from the strengths and evolution of Langchain, Agno, OpenAI Agents SDK, and Google ADK. Key architectural elements include a multi-package structure, stable core abstractions based on a Runnable interface, an explicit graph-based orchestration engine for handling complex agent logic and state, a flexible memory system with support for persistence, defined patterns for multi-agent interaction, and a strong emphasis on developer experience through integrated observability and evaluation tools.

B. Next Steps for Development

The immediate next steps to initiate the development process based on this plan are:

1. **Detailed API Design:** Flesh out the specific methods, parameters, and return types for the core interfaces defined in sdk-core (BaseRunnable, BaseChatModel, BaseTool, BaseMemory, etc.).
2. **Repository Setup:** Create the initial monorepo structure with the proposed packages (sdk-core, sdk-agents, etc.) and set up basic CI/CD pipelines.
3. **Core Engine Prototyping:** Begin implementing the foundational elements of the graph-based orchestration engine (StateGraph, node execution, basic state management) in sdk-agents.
4. **Implement Core Abstractions:** Develop the concrete implementations for essential core components like message types and the Document class.
5. **Establish Contribution Guidelines:** Define processes for code style, testing,

documentation, and contributions to prepare for future ecosystem growth.

By following this plan, the development team can build a powerful, flexible, and developer-friendly agent framework SDK capable of supporting a wide range of sophisticated AI applications.

Works cited

1. Introduction | 🦜 LangChain, accessed April 21, 2025, <https://python.langchain.com/docs/introduction/>
2. Architecture | 🦜 LangChain, accessed April 21, 2025, <https://python.langchain.com/docs/concepts/architecture/>
3. Conceptual guide | 🦜 LangChain, accessed April 21, 2025, <https://python.langchain.com/v0.2/docs/concepts/>
4. Guide to LangChain Runnable Architecture - DZone, accessed April 21, 2025, <https://dzone.com/articles/guide-to-langchain-runnable-architecture>
5. LangChain Expression Language Explained - Pinecone, accessed April 21, 2025, <https://www.pinecone.io/learn/series/langchain/langchain-expression-language/>
6. LangChain Expression Language (LCEL), accessed April 21, 2025, <https://python.langchain.com/docs/concepts/lcel/>
7. LangChain Python API Reference, accessed April 21, 2025, https://python.langchain.com/api_reference/
8. LangChain Python API Reference, accessed April 21, 2025, https://python.langchain.com/v0.2/api_reference/
9. langchain-ai/langchain: Build context-aware reasoning applications - GitHub, accessed April 21, 2025, <https://github.com/langchain-ai/langchain>
10. AutoGen vs LangChain: Comparison for LLM Applications - PromptLayer, accessed April 21, 2025, <https://blog.promptlayer.com/autogen-vs-langchain/>
11. Overview - LangChain, accessed April 21, 2025, https://python.langchain.com/docs/versions/v0_2/overview/
12. Towards LangChain 0.1: LangChain-Core and LangChain-Community - LangChain Blog, accessed April 21, 2025, <https://blog.langchain.dev/the-new-langchain-architecture-langchain-core-v0-1-langchain-community-and-a-path-to-langchain-v0-1/>
13. LangSmith - LangChain, accessed April 21, 2025, <https://www.langchain.com/langsmith>
14. Get started with LangSmith | 🦜🔧 LangSmith, accessed April 21, 2025, <https://docs.smith.langchain.com/>
15. agno · PyPI, accessed April 21, 2025, <https://pypi.org/project/agno/1.0.5/>
16. Unleashing the Power of Agno: Building Multi-Modal Agents with a Lightweight Python Library - RPABOTS.WORLD, accessed April 21, 2025, <https://rpabotsworld.com/agno-building-multi-modal-agents-with-a-lightweight-python-library/>
17. Agno is a lightweight library for building Agents with memory, knowledge, tools and reasoning. - GitHub, accessed April 21, 2025,

- <https://github.com/agno-agi/agno>
18. Agno Framework: A Lightweight Library for Building Multimodal Agents - Analytics Vidhya, accessed April 21, 2025, <https://www.analyticsvidhya.com/blog/2025/03/agno-framework/>
 19. Building a Multi-Agent Financial News System with Agno and Agent UI - Marketcalls, accessed April 21, 2025, <https://www.marketcalls.in/python/building-a-multi-agent-financial-news-system-with-agno-and-agent-ui.html>
 20. Agno - Weaviate, accessed April 21, 2025, <https://weaviate.io/developers/integrations/llm-agent-frameworks/agno>
 21. Agent - Agno, accessed April 21, 2025, <https://docs.agno.com/reference/agents/agent>
 22. agno-agi/agent-ui: A modern chat interface for AI agents built with Next.js, Tailwind CSS, and TypeScript. - GitHub, accessed April 21, 2025, <https://github.com/agno-agi/agent-ui>
 23. openai/openai-agents-python: A lightweight, powerful ... - GitHub, accessed April 21, 2025, <https://github.com/openai/openai-agents-python>
 24. A practical guide to building agents | OpenAI, accessed April 21, 2025, <https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf>
 25. New tools for building agents | OpenAI, accessed April 21, 2025, <https://openai.com/index/new-tools-for-building-agents/>
 26. cookbook/gen-ai/openai/agents-sdk-intro.ipynb at main - GitHub, accessed April 21, 2025, <https://github.com/aurelio-labs/cookbook/blob/main/gen-ai/openai/agents-sdk-intro.ipynb>
 27. Handoffs - OpenAI Agents SDK, accessed April 21, 2025, <https://openai.github.io/openai-agents-python/handoffs/>
 28. Building Agentic AI Applications using OpenAI Agents SDK - Association of Data Scientists, accessed April 21, 2025, <https://adasci.org/building-agentic-ai-applications-using-openai-agents-sdk/>
 29. Agent Development Kit - Google, accessed April 21, 2025, <https://google.github.io/adk-docs/>
 30. google/adk-docs: An open-source, code-first Python toolkit for building, evaluating, and deploying sophisticated AI agents with flexibility and control. - GitHub, accessed April 21, 2025, <https://github.com/google/adk-docs>
 31. Introduction to Conversational Context: Session, State, and Memory - Agent Development Kit - Google, accessed April 21, 2025, <https://google.github.io/adk-docs/sessions/>
 32. Memory - Agent Development Kit - Google, accessed April 21, 2025, <https://google.github.io/adk-docs/sessions/memory/>
 33. accessed January 1, 1970, https://google.github.io/adk-docs/sessions_memory/
 34. Getting Started with Google ADK: Build Agentic AI Systems with Ease - Sanket Daru, accessed April 21, 2025, <https://sanketdaru.com/blog/google-adk-agentic-ai-framework-getting-started/>

35. Unlock Advanced AI: An Introduction to Google's Agent Development Kit (ADK), accessed April 21, 2025,
<https://wowdata.science/unlock-advanced-ai-an-introduction-to-googles-agent-development-kit-adk/>
36. Why Evaluate Agents - Agent Development Kit - Google, accessed April 21, 2025,
<https://google.github.io/adk-docs/evaluate/>
37. LangGraph - LangChain Blog, accessed April 21, 2025,
<https://blog.langchain.dev/langgraph/>
38. Unleashing the Power of LangChain Expression Language (LCEL): from proof of concept to production - Artefact, accessed April 21, 2025,
<https://www.artefact.com/blog/unleashing-the-power-of-langchain-expression-language-lcel-from-proof-of-concept-to-production/>
39. Langchain Python API Reference, accessed April 21, 2025,
<https://api.python.langchain.com/>
40. LangChain Expression Language (LCEL), accessed April 21, 2025,
<https://js.langchain.com/docs/concepts/lcel/>
41. tools — LangChain documentation - LangChain, accessed April 21, 2025,
https://python.langchain.com/api_reference/core/tools.html
42. Develop an Agent Development Kit agent | Generative AI on Vertex AI - Google Cloud, accessed April 21, 2025,
<https://cloud.google.com/vertex-ai/generative-ai/docs/agent-engine/develop/adk>
43. agents — LangChain documentation, accessed April 21, 2025,
https://python.langchain.com/api_reference/langchain/agents.html
44. Introduction - Agno, accessed April 21, 2025,
<https://docs.agno.com/tools/introduction>
45. documents - LangChain, accessed April 21, 2025,
https://python.langchain.com/api_reference/core/documents.html
46. LangChain documentation, accessed April 21, 2025,
https://python.langchain.com/api_reference/core/documents/langchain_core_documents.base.Document.html
47. How to create a langchain doc from an str? - Stack Overflow, accessed April 21, 2025,
<https://stackoverflow.com/questions/76551067/how-to-create-a-langchain-doc-from-an-str>
48. GitHub - LangChain, accessed April 21, 2025,
https://python.langchain.com/docs/integrations/document_loaders/github/
49. GitHub | 🦜 Langchain, accessed April 21, 2025,
https://js.langchain.com/docs/integrations/document_loaders/web_loaders/github/
50. Agents | 🦜 LangChain, accessed April 21, 2025,
<https://python.langchain.com/docs/concepts/agents/>
51. Complete Guide to Building LangChain Agents with the LangGraph Framework - Zep, accessed April 21, 2025,
<https://www.getzep.com/ai-agents/langchain-agents-langgraph>
52. How to manage agent steps - GitHub Pages, accessed April 21, 2025,

- <https://langchain-ai.github.io/langgraphjs/how-tos/managing-agent-steps/>
53. LangGraph Tutorial: Building LLM Agents with LangChain's Agent Framework - Zep, accessed April 21, 2025, <https://www.getzep.com/ai-agents/langgraph-tutorial>
 54. LangChain Expression Language (LCEL), accessed April 21, 2025, https://python.langchain.com/v0.1/docs/expression_language/
 55. Introduction to LangGraph - LangChain Academy, accessed April 21, 2025, <https://academy.langchain.com/courses/intro-to-langgraph>
 56. LangGraph - LangChain, accessed April 21, 2025, <https://www.langchain.com/langgraph>
 57. memory — LangChain documentation, accessed April 21, 2025, https://python.langchain.com/api_reference/langchain/memory.html
 58. memory — LangChain documentation, accessed April 21, 2025, <https://api.python.langchain.com/en/latest/langchain/memory.html>
 59. ConversationBufferMemory — LangChain documentation, accessed April 21, 2025, https://python.langchain.com/api_reference/langchain/memory/langchain.memory.buffer.ConversationBufferMemory.html
 60. agno/cookbook/examples/agents/agno_support_agent.py at main - GitHub, accessed April 21, 2025, https://github.com/agno-agi/agno/blob/main/cookbook/examples/agents/agno_support_agent.py
 61. agno/cookbook/examples/agents/deep_knowledge.py at main - GitHub, accessed April 21, 2025, https://github.com/agno-agi/agno/blob/main/cookbook/examples/agents/deep_knowledge.py
 62. agno/cookbook/examples/agents/agno_assist.py at main - GitHub, accessed April 21, 2025, https://github.com/agno-agi/agno/blob/main/cookbook/examples/agents/agno_assist.py
 63. [Bug] Memory leak when making multiple queries one after another · Issue #2632 - GitHub, accessed April 21, 2025, <https://github.com/agno-agi/agno/issues/2632>
 64. Agno - GroqCloud, accessed April 21, 2025, <https://console.groq.com/docs/agno>
 65. LangChain Academy - GitHub, accessed April 21, 2025, <https://github.com/langchain-ai/langchain-academy>
 66. langchain · PyPI, accessed April 21, 2025, <https://pypi.org/project/langchain/>
 67. A collection of sample agents built with Agent Development (ADK) - GitHub, accessed April 21, 2025, <https://github.com/google/adk-samples>
 68. Add observability to your LLM application | 🦜🔧 LangSmith - LangChain, accessed April 21, 2025, <https://docs.smith.langchain.com/observability/tutorials/observability>
 69. Introducing End-to-End OpenTelemetry Support in LangSmith - LangChain Blog, accessed April 21, 2025, <https://blog.langchain.dev/end-to-end-opentelemetry-langsmith/>
 70. Deploying Your Agent - Agent Development Kit - Google, accessed April 21,

- 2025, <https://google.github.io/adk-docs/deploy/>
71. RunnableWithFallbacks — LangChain documentation, accessed April 21, 2025, https://python.langchain.com/api_reference/core/runnables/langchain_core.runnables.fallbacks RunnableWithFallbacks.html
 72. Fallbacks - LangChain.js, accessed April 21, 2025, https://js.langchain.com/docs/how_to/fallbacks/
 73. Fallbacks | 🦉 LangChain, accessed April 21, 2025, <https://python.langchain.com/v0.1/docs/guides/productionization/fallbacks/>